Session 21&22

Generative Adversarial Network (GAN)

Deep Learning | Hobot | Zahra Amini

Hobot Academy

Telegram: @zahraamini_ai & Instagram:@zahraamini_ai & LinkedIn: @zahraamini-ai

https://aminizahra.github.io

یه داستان معروف:

یک دزد و یک پلیس وجود دارند که هر دو در یک شهر زندگی می‌کنند. دزد تلاش می‌کند که کالاهای تقلبی تولید کند و آن‌ها را به فروش برساند، به این امید که کسی متوجه تقلبی بودن آن‌ها نشود. از طرف دیگر، پلیس به دنبال دستگیری دزد است و تمام تلاشش را می‌کند تا بتواند کالای تقلبی را از کالای واقعی تشخیص دهد و دزد را دستگیر کند

دزد در ابتدا کالاهای تقلبی خود را با کیفیت پایین تولید می‌کند، اما هر بار که پلیس موفق به شناسایی تقلبی بودن کالاها می‌شود، دزد از این تجربه استفاده می‌کند تا در دفعات بعدی کالاهای تقلبی بهتری تولید کند که شبیه‌تر به کالاهای واقعی باشند. از طرف دیگر، پلیس نیز هر بار با مشاهده تلاش‌های دزد، مهارت‌های خود را بهبود می‌بخشد تا بتواند دزد را بهتر شناسایی کند

این ماجرا همچنان ادامه دارد و هر بار دزد تلاش می‌کند که کالای تقلبی بهتری تولید کند، و پلیس هم در تلاش است که با مهارت‌های بهبود یافته خود، تقلب دزد را کشف کند

معماری مدل GAN از دو جزء اساسی زیر تشکیل شده:

Generator

Discrimin

? حالا به نظر شما، در داستان بالا کدام نقش مربوط به مولد و کدام مربوط به متمایزکننده است؟

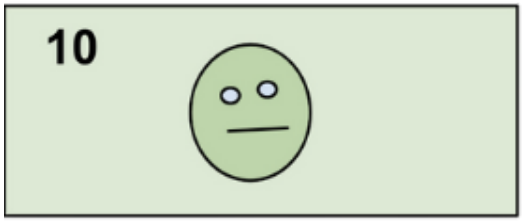
Bad GAN

Generated Data | Discriminator | Real Data

FAKE

REAL

OK GAN



FAKE

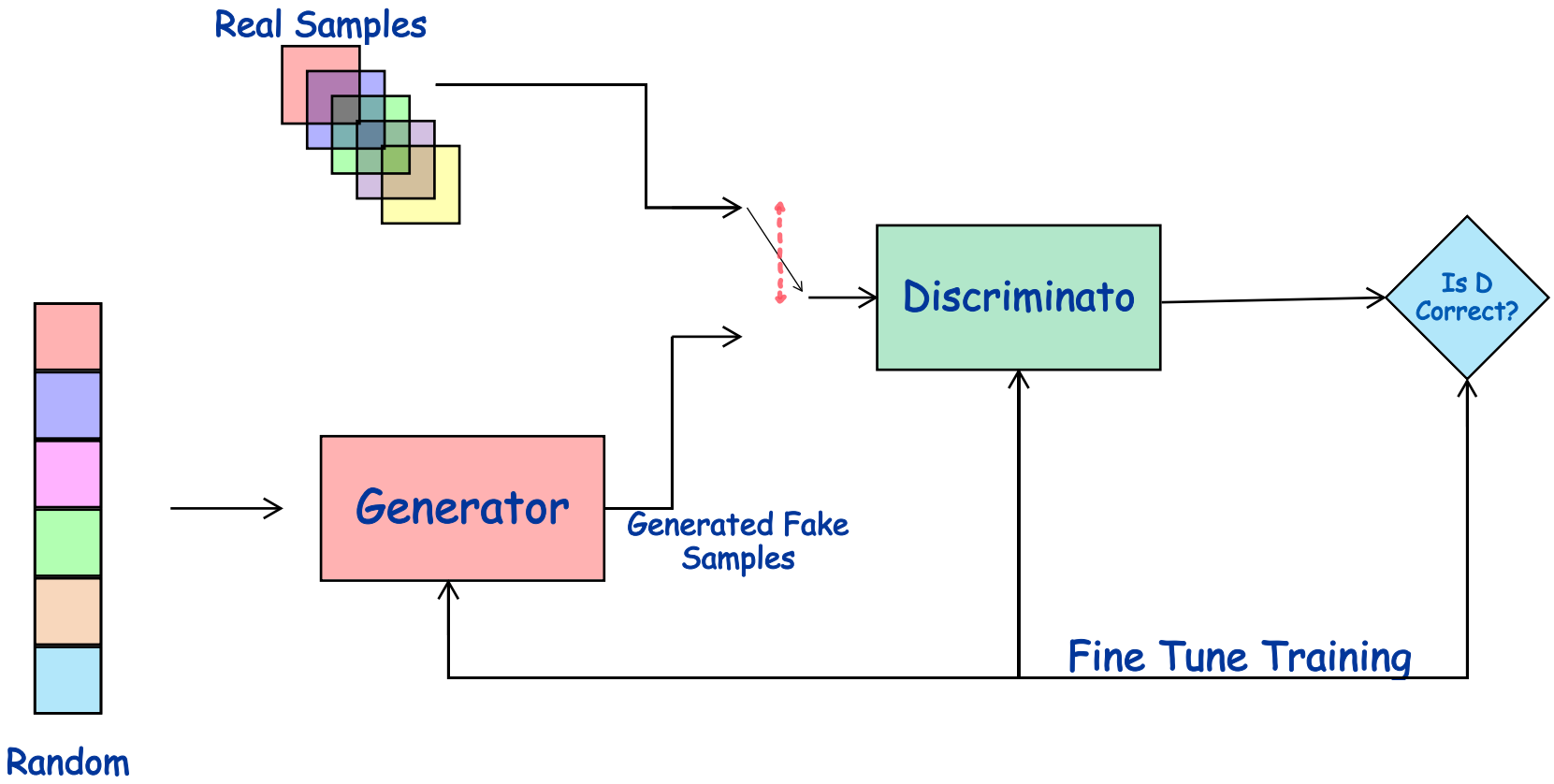
REAL

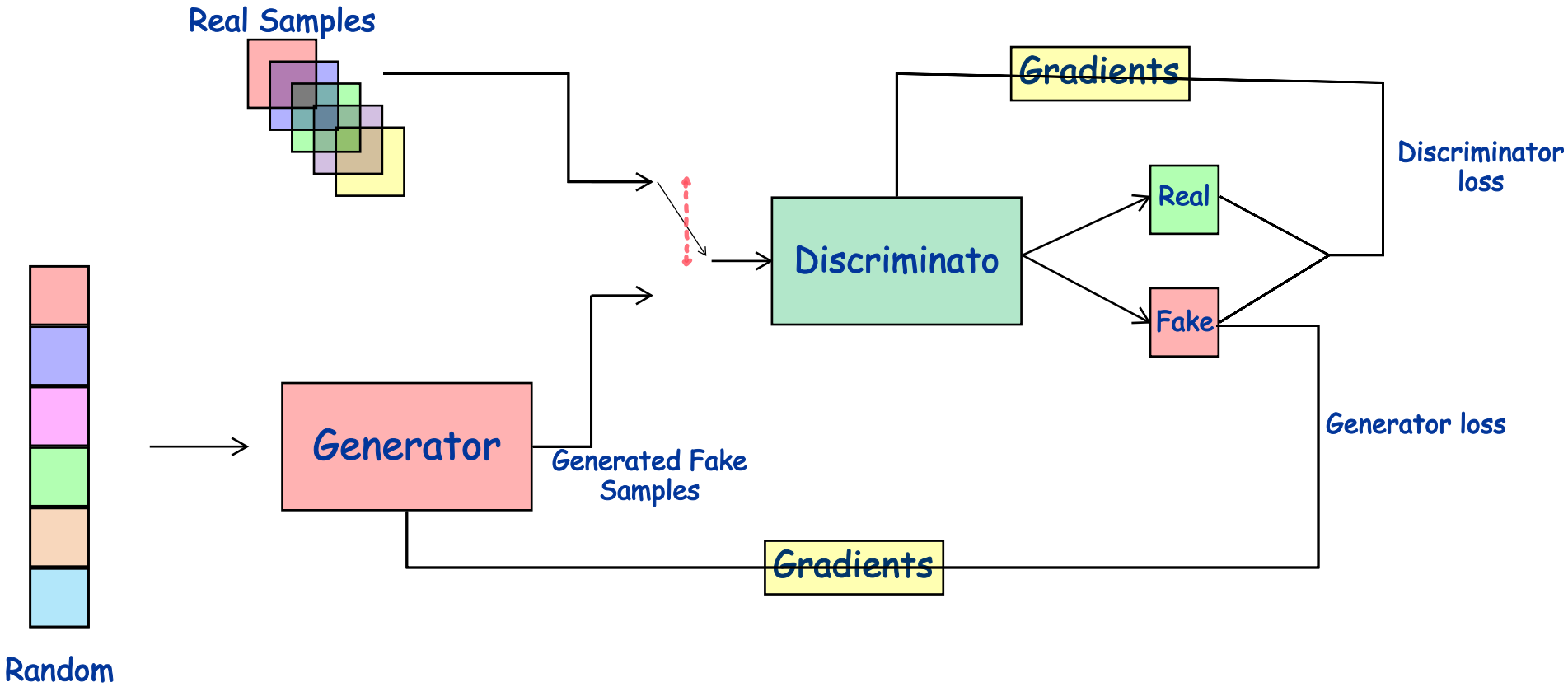
Good GAN

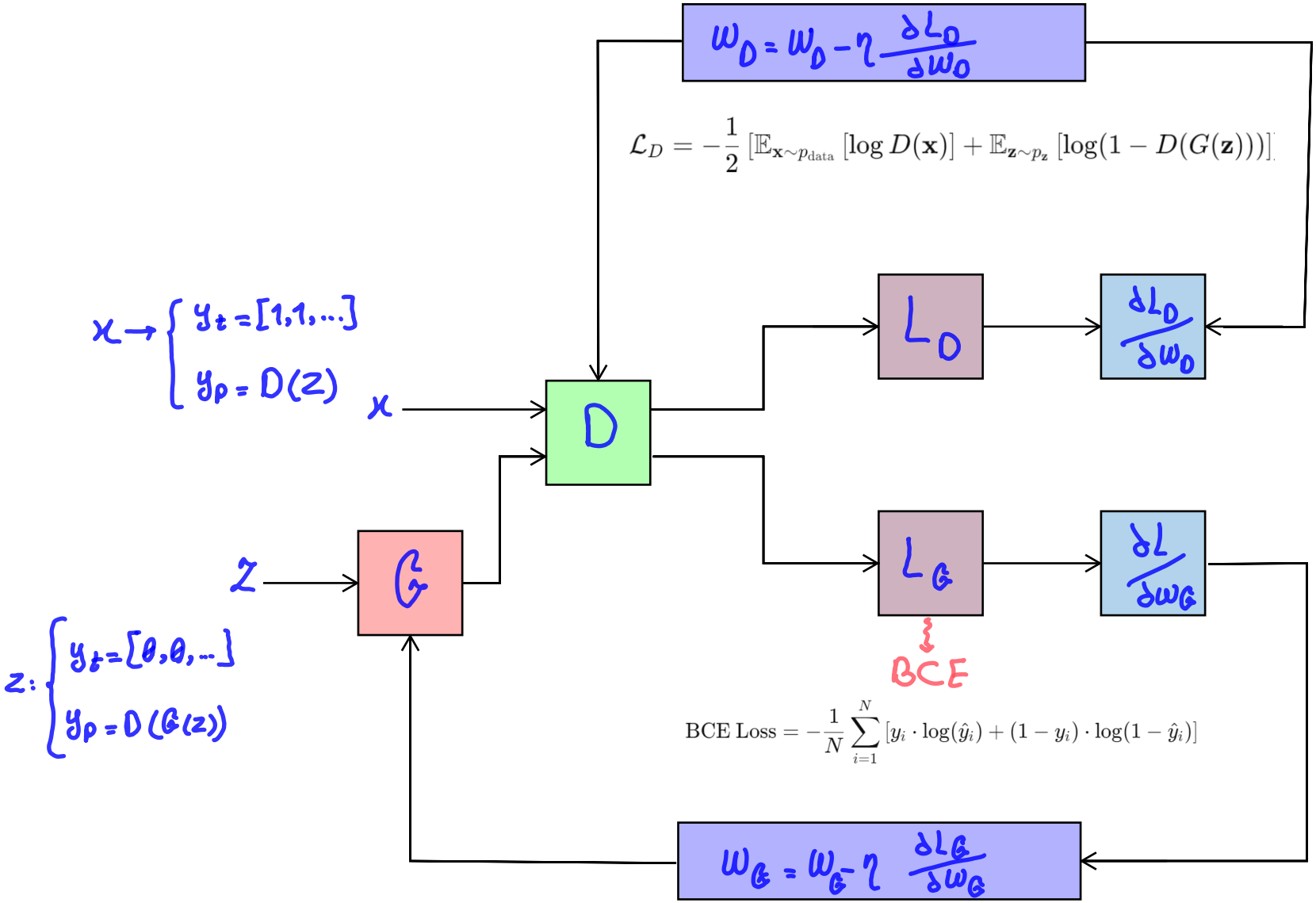
REAL

REAL

GAN Architecture




Real Samples
Gradients
Discriminator loss
Real
Discriminato
Fake
Generator loss
Generator
Generated Fake Samples
Gradients
Random



شبکه	لاس پایین	لاس بالا
Discriminator (تمیزدهنده)	تمیزدهنده به خوبی داده‌های واقعی را از داده‌های جعلی تشخیص می‌دهد.	تمیزدهنده نمی‌تواند به خوبی داده‌های جعلی را شناسایی کند؛ مولد داده‌های شبیه به داده‌های واقعی تولید کرده است.
Generator (مولد)	مولد توانسته است داده‌هایی تولید کند که تمیزدهنده را فریب دهد.	مولد نتوانسته است داده‌های خوبی تولید کند؛ تمیزدهنده به راحتی داده‌های جعلی را تشخیص می‌دهد.

GAN pseudocode

```
# Step 1: Initialize the Generator and Discriminator models
Initialize Generator model G
Initialize Discriminator model D

# Step 2: Define the loss functions and optimizers
Define Binary Cross-Entropy loss as loss_function
Define optimizers for G and D

# Step 3: Training Loop
for each epoch in range(num_epochs):

    for each batch in training_data:

        # Step 3.1: Train the Discriminator
        # Generate fake data using the Generator
        noise = random_noise_vector(batch_size)
        fake_data = G(noise)

        # Get real data
        real_data = get_real_data(batch)

        # Discriminator loss on real data
        real_output = D(real_data)
        real_loss = loss_function(real_output, real_labels)

        # Discriminator loss on fake data
        fake_output = D(fake_data.detach()) # detach to avoid training G with D
        fake_loss = loss_function(fake_output, fake_labels)

        # Total Discriminator loss
        D_loss = real_loss + fake_loss

        # Backpropagation and optimization for Discriminator
        D_optimizer.zero_grad()
        D_loss.backward()
        D_optimizer.step()

        # Step 3.2: Train the Generator
        # Generate fake data again
        noise = random_noise_vector(batch_size)
        fake_data = G(noise)

        # Generator loss
        fake_output = D(fake_data)
        G_loss = loss_function(fake_output, real_labels) # want D to think fake is real

        # Backpropagation and optimization for Generator
        G_optimizer.zero_grad()
        G_loss.backward()
        G_optimizer.step()

    # Optional: Print loss values, save model checkpoints, or generate sample outputs

# Step 4: Model saving after training
Save the trained Generator model
```

1. Discriminator Loss

The discriminator's objective is to correctly classify real and fake images. It does so by maximizing the following objective:

$$\text{Loss}_D = -\left(\mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]\right)$$

Where:

- $D(x)$ is the discriminator's estimate of the probability that real data instance x is real.
- $G(z)$ is the generator's output when given noise z.
- $p_{\text{data}}(x)$ is the distribution of real data.
- $p_z(z)$ is the distribution of the noise input to the generator.

This can be rewritten as:

$$\text{Loss}_D = -\mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] - \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

$D(x)$ تخمین متمایزکننده از احتمال واقعی بودن داده x است.

$G(z)$ خروجی مولد وقتی به آن نویز z داده می‌شود.

$p_{\text{data}}(x)$ توزیع داده‌های واقعی است.

$p_z(z)$ توزیع نویزی است که به مولد داده می‌شود.

2. Generator Loss

The generator's objective is to produce images that the discriminator cannot distinguish from real images. The generator tries to minimize the following objective, which is essentially trying to maximize the discriminator's mistake:

$$\text{Loss}_G = -\mathbb{E}_{z \sim p_z(z)}[\log(D(G(z)))]$$

Where:

- $D(G(z))$ is the discriminator's estimate of the probability that the fake data instance $G(z)$ is real.

This loss encourages the generator to produce outputs that are classified as real by the discriminator.

3. Combined Loss (Minimax Game)

The GAN training process can be seen as a two-player minimax game where the generator tries to minimize the loss while the discriminator tries to maximize it:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$$

- The discriminator D aims to maximize this value, meaning it tries to correctly classify real and fake images.
- The generator G aims to minimize this value, meaning it tries to produce images that the discriminator classifies as real.

The accuracy A_D of the discriminator can then be calculated as:

$$A_D = \frac{\text{Number of Correct Predictions}}{\text{Total Number of Samples}} = \frac{\text{Correct Real Classifications} + \text{Correct Fake}}{N_{\text{real}} + N_{\text{fake}}}$$

Where:

- The number of correct real classifications is:

$$\text{Correct Real Classifications} = \sum_{i=1}^{N_{\text{real}}} 1[D(x^i_{\text{real}}) \geq 0.5]$$

  Here, $1[D(x^i_{\text{real}}) \geq 0.5]$ is an indicator function that equals 1 if $D(x^i_{\text{real}}) \geq 0.5$ and 0 otherwise.

- The number of correct fake classifications is:

$$\text{Correct Fake Classifications} = \sum_{j=1}^{N_{\text{fake}}} 1[D(G(z^j)) < 0.5]$$

  Here, $1[D(G(z^j)) < 0.5]$ is an indicator function that equals 1 if $D(G(z^j)) < 0.5$ and 0 otherwise.

N_{real}: تعداد نمونه‌های واقعی.

N_{fake}: تعداد نمونه‌های تقلبی.

$D(x_{\text{real}})$: خروجی متمایزکننده برای یک نمونه واقعی x_{real}.

$D(G(z))$: خروجی متمایزکننده برای یک نمونه تقلبی $G(z)$.

فرض می‌کنیم متمایزکننده یک پیش‌بینی درست انجام می‌دهد اگر:

$0.5 \leq D(x_{\text{real}})$ (به درستی به عنوان واقعی طبقه‌بندی شده باشد).

$0.5 > D(G(z))$ (به درستی به عنوان تقلبی طبقه‌بندی شده باشد).

Step 1: Define the Generator Network

- **Input:** A random noise vector $\mathbf{z} = [0.1, 0.2, 0.3, 0.4]$
- **Hidden Layer:**
  - Number of nodes: 4
  - Activation function: ReLU
  - Weights:

$$\mathbf{W}_{\text{hidden}} = \begin{bmatrix} 0.1 & -0.2 & 0.3 & 0.4 \\ -0.4 & 0.2 & -0.1 & 0.2 \\ 0.2 & 0.4 & -0.3 & 0.1 \\ 0.3 & -0.3 & 0.2 & 0.2 \end{bmatrix}$$

  - Biases: $\mathbf{b}_{\text{hidden}} = [0.1, 0.2, 0.1, 0.2]$
- **Output Layer:**
  - Number of nodes: 9
  - Activation function: Sigmoid
  - Weights:

$$\mathbf{W}_{\text{output}} = \begin{bmatrix} 0.2 & -0.1 & 0.3 & 0.4 \\ -0.3 & 0.1 & 0.1 & 0.2 \\ -0.4 & 0.3 & 0.1 & 0.2 \end{bmatrix}$$

  - Biases: $\mathbf{b}_{\text{output}} = [0.1, 0.2, 0.3]$

Step 2: Calculate the Generator Output

Output of the Hidden Layer:

1. Calculate the weighted sum and add biases:

$$\mathbf{hidden_output} = \mathbf{W}_{\text{hidden}} \cdot \mathbf{z} + \mathbf{b}_{\text{hidden}}$$

$$\mathbf{hidden_output} = \begin{bmatrix} 0.1 \times 0.1 - 0.04 + 0.09 + 0.16 + 0.1 \\ -0.04 + 0.04 - 0.03 + 0.08 + 0.2 \\ 0.02 + 0.08 - 0.09 + 0.04 + 0.1 \\ 0.03 - 0.06 + 0.06 + 0.08 + 0.2 \end{bmatrix}$$

$$\mathbf{hidden_output} = \begin{bmatrix} 0.41 \\ 0.25 \\ 0.15 \\ 0.31 \end{bmatrix}$$

2. Apply ReLU activation function:

$$\mathbf{hidden_output} = \text{ReLU}(\mathbf{hidden_output}) = \begin{bmatrix} 0.41 \\ 0.25 \\ 0.15 \\ 0.31 \end{bmatrix}$$

Output of the Output Layer:

1. Calculate the weighted sum and add biases:

$$\mathbf{output} = \mathbf{W}_{\text{output}} \cdot \mathbf{hidden_output} + \mathbf{b}_{\text{output}}$$

$$\mathbf{output} = \begin{bmatrix} 0.082 + (-0.025) + 0.045 + 0.124 + 0.1 \\ -0.123 + 0.025 + 0.015 + 0.062 + 0.2 \\ -0.164 + 0.075 + 0.015 + 0.062 + 0.3 \end{bmatrix}$$

$$\mathbf{output} = \begin{bmatrix} 0.486 \\ 0.692 \\ 0.494 \end{bmatrix}$$

2. Apply Sigmoid activation function:

$$\mathbf{output} = \text{Sigmoid}(\mathbf{output}) = \begin{bmatrix} \frac{1}{1+e^{-0.486}} \\ \frac{1}{1+e^{-0.692}} \\ \frac{1}{1+e^{-0.494}} \end{bmatrix}$$

$$\mathbf{output} \approx \begin{bmatrix} 0.618 \\ 0.666 \\ 0.621 \end{bmatrix}$$

Step 3: Define and Calculate the Discriminator Output

Define the Discriminator Network:

- **Input:** A 3x3 image (9 pixels), either generated by the generator or a real image. The input to the network is a 9-dimensional vector: $\mathbf{x} = [x_1, x_2, \ldots, x_9]$.
- **Hidden Layer:**
  - Number of nodes: 4
  - Activation function: ReLU
  - Weights:

$$\mathbf{W}_{\text{hidden}} = \begin{bmatrix} 0.2 & 0.3 & -0.1 & 0.4 & -0.4 & 0.1 & 0.2 & -0.3 & 0.3 \\ -0.2 & 0.1 & 0.4 & -0.1 & 0.2 & 0.3 & -0.4 & 0.1 & -0.2 \\ 0.3 & -0.2 & 0.1 & 0.3 & -0.1 & -0.4 & 0.2 & 0.4 & 0.1 \\ 0.4 & -0.3 & 0.2 & -0.4 & 0.3 & -0.1 & 0.1 & -0.2 & 0.2 \end{bmatrix}$$

  - Biases: $\mathbf{b}_{\text{hidden}} = [0.1, -0.1, 0.2, 0.3]$

- **Output Layer:**
  - Number of nodes: 1 (to decide whether the image is real or fake)
  - Activation function: Sigmoid
  - Weights:

$$\mathbf{W}_{\text{output}} = \begin{bmatrix} 0.2 \\ -0.3 \\ 0.4 \\ -0.2 \end{bmatrix}$$

  - Bias: $b_{\text{output}} = 0.1$

Calculate the Discriminator Output:

1. Hidden Layer Output:

$$\mathbf{hidden_output} = \mathbf{W}_{\text{hidden}} \cdot \mathbf{x} + \mathbf{b}_{\text{hidden}}$$

Assume $\mathbf{x} = [0.6, 0.7, 0.8, 0.9, 0.1, 0.2, 0.3, 0.4, 0.5]$.

$$\mathbf{hidden_output} = \begin{bmatrix} 0.12 + 0.21 - 0.08 + 0.36 - 0.04 + 0.02 + 0.06 - 0.12 + 0.15 + 0.1 \\ -0.12 + 0.07 + 0.32 - 0.09 + 0.02 + 0.06 - 0.12 + 0.04 - 0.1 - 0.1 \\ 0.18 - 0.14 + 0.08 + 0.27 - 0.01 - 0.08 + 0.06 + 0.16 + 0.05 + 0.2 \\ 0.24 - 0.21 + 0.16 - 0.36 + 0.03 - 0.02 + 0.03 - 0.08 + 0.1 + 0.3 \end{bmatrix}$$

$$\mathbf{hidden_output} = \begin{bmatrix} 0.78 \\ -0.16 \\ 0.57 \\ 0.19 \end{bmatrix}$$

2. Apply ReLU activation function:

$$\mathbf{hidden_output} = \mathrm{ReLU}(\mathbf{hidden_output}) = \begin{bmatrix} 0.78 \\ 0 \\ 0.57 \\ 0.19 \end{bmatrix}$$

3. Output Layer:

$$\mathrm{output} = \mathrm{Sigmoid}(\mathbf{W}_{\mathrm{output}} \cdot \mathbf{hidden_output} + b_{\mathrm{output}})$$

$$\mathrm{output} = \mathrm{Sigmoid}\left(0.2 \times 0.78 - 0.3 \times 0 + 0.4 \times 0.57 - 0.2 \times 0.19 + 0.1\right)$$

$$\mathrm{output} = \mathrm{Sigmoid}\left(0.156 + 0 + 0.228 - 0.038 + 0.1\right)$$

$$\mathrm{output} = \mathrm{Sigmoid}\left(0.446\right)$$

4. Apply Sigmoid activation function:

$$\mathrm{output} = \frac{1}{1 + e^{-0.446}} \approx 0.609$$

Step 4: Calculate the Discriminator Loss

Discriminator Loss for Real Images:

Assume the discriminator gives $D(x_{\text{real}}) = 0.9$ for a real image:

$$\text{Loss}_D(\text{real}) = -\log(D(x_{\text{real}})) = -\log(0.9) \approx 0.105$$

Discriminator Loss for Fake Images:

Assume the discriminator gives $D(G(\mathbf{z})) = 0.618$ for an image generated by the generator:

$$\text{Loss}_D(\text{fake}) = -\log(1 - D(G(\mathbf{z}))) = -\log(1 - 0.618) \approx 0.961$$

Total Discriminator Loss:

The total discriminator loss is the average of the losses for real and fake images:

$$\text{Loss}_D = \frac{1}{2}(\text{Loss}_D(\text{real}) + \text{Loss}_D(\text{fake})) = \frac{1}{2}(0.105 + 0.961) \approx 0.533$$

Step 5: Calculate the Generator Loss

Step 5: Calculate the Generator Loss

The generator aims to make the discriminator believe that the fake images are real. The generator loss is calculated as:

$$\text{Loss}_G = -\log(D(G(\mathbf{z}))) = -\log(0.618) \approx 0.482$$

Step 6.1: Accuracy Calculation for the Discriminator

The discriminator produces outputs between 0 and 1, where values closer to 1 indicate that the discriminator thinks the image is real, and values closer to 0 indicate that the discriminator thinks the image is fake.

Let's assume we set a threshold of 0.5 to classify the output:

- If $D(x) \geq 0.5$, the discriminator classifies the image as real.
- If $D(x) < 0.5$, the discriminator classifies the image as fake.

To calculate accuracy, we compare the discriminator's predictions against the ground truth (real or fake) and calculate the percentage of correct predictions.

Step 6.2: Example Calculation

Assume the following:

- $D(x_{\text{real}}) = 0.9$ for a real image.
- $D(G(\mathbf{z})) = 0.618$ for a generated image.

Predictions:

- For the real image: $D(x_{\text{real}}) = 0.9 \geq 0.5$ (Correctly classified as real)
- For the fake image: $D(G(\mathbf{z})) = 0.618 \geq 0.5$ (Incorrectly classified as real)

If we have 50 real and 50 fake images:

- Suppose the discriminator correctly classifies 45 real and 40 fake images.
- Accuracy = $\frac{45+40}{50+50} = 0.85$ or 85%.

Update Weights ---> Step 7.1: Gradient Calculation

For simplicity, we'll use a basic gradient descent approach:

- For the discriminator, we minimize the loss function:

$$\text{Loss}_D = -\left(\log(D(x_{\text{real}})) + \log(1 - D(G(\mathbf{z})))\right)$$

- For the generator, we minimize the loss function:

$$\text{Loss}_G = -\log(D(G(\mathbf{z})))$$

Step 7.2: Weight Update Formula

The weights are updated as follows:

- For discriminator weights W_D:

$$W_D = W_D - \eta \cdot \frac{\partial \text{Loss}_D}{\partial W_D}$$

- For generator weights W_G:

$$W_G = W_G - \eta \cdot \frac{\partial \text{Loss}_G}{\partial W_G}$$

where η is the learning rate.

Step 7.3: Calculation

Let's assume:

- Learning rate $\eta = 0.01$.
- We have computed gradients $\frac{\partial \text{Loss}_D}{\partial W_D} = 0.05$ and $\frac{\partial \text{Loss}_G}{\partial W_G} = 0.04$.

Then:

- Update discriminator weights:

$$W_D = W_D - 0.01 \times 0.05$$

- Update generator weights:

$$W_G = W_G - 0.01 \times 0.04$$

This process continues iteratively over multiple epochs to improve the performance of both networks.

https://poloclub.github.io/ganlab/